

Ressource R101 – Initiation aux réseaux informatiques

Cours #1 : Codage de l'information

B.U.T Réseaux et Télécoms

Année 2025-2026



IUT R&T de Béthune

1. Le système binaire
2. Le changement de base
3. Les opérations binaires
4. Le codage du texte

- ♦ L' informatique est le traitement automatique de l' information.
 - ❑ Automatique → réalisable par une machine
 - ❑ Information → données de l'utilisateur (des nombres, du son, des images, du texte ...)
- ♦ Pour travailler sur des données, l'ordinateur a besoin de les représenter dans un langage qu'il comprend.
- ♦ Le langage de la machine est basé sur le **système binaire**.
- ♦ On appelle **codage** le procédé qui consiste à traduire les données en langage machine et **décodage** le procédé inverse.

◆ Définition

La numération, c'est la façon d'énoncer ou d'écrire des nombres.

◆ Exemple :

□ La numération Romaine:

- ♦ I, II, III, IV, V, VI, VII, VIII, IX, X, L, C, M
- ♦ MMXXV c'est ?

□ la numération Arabe :

- ♦ 0, 1, 2, 3, 4, 5, 6, 7, 8 et 9.

◆ C'est aussi un système (appelé base) qui organise la suite des nombres en séries hiérarchisées.

- ◆ Le système décimal:

Il est basé sur le nombre 10 (10 étant la base du système décimal).

Les chiffres 0, 1, ..., 8, 9, soit 10 caractères au total sont utilisés pour la représentation des nombres.

Un nombre est décomposable en puissances de 10, que l'on énonce en allant de la plus élevée à la plus faible.

Exemple :

$$3528 = 3 \cdot 10^3 + 5 \cdot 10^2 + 2 \cdot 10^1 + 8 \cdot 10^0$$

- ◆ On note en indice la base du système de numération dans laquelle le nombre est représenté.
 - ◆ Exemple : 3528_{10} ou $(3528)_{10}$ ou 3528
- ◆ Pour le nombre décimal 3528, chaque chiffre possède un rang.
- ◆ Le chiffre **3** a le rang des milliers, on dit que son "poids " est de mille. Pour le nombre **3528**, le chiffre **3** est appelé le chiffre de poids fort, et le **8** le chiffre de poids faible. Les anglo-saxons parlent aussi de 'digit'.
- ◆ Conclusion: un nombre quelconque dans une base donnée peut s'écrire:
 - ◆ $a_{n-1} \dots a_1 a_0 = a_{n-1} * b^{n-1} + \dots + a_1 * b^1 + a_0 * b^0$
 - ◆ b étant appelé la base

- ▶ Le bit est la plus petite unité de représentation de l'information.
- ▶ Bit est la concaténation de **binary digit** (chiffre binaire)
- ▶ Un bit a pour valeur 0 ou 1.
- ▶ Dans certains cas (opérateurs logiques), on associe :
 - 0 à faux (false)
 - 1 à vrai (true)
- ▶ Une suite de 8 bits est appelée un **octet**.
- ▶ Une suite de 4 bits est appelée un **quartet**.
- ▶ Une suite de 16 bits est appelée un **mot**.
- ▶ En informatique, on code des données avec des séquences de bits (ou nombres binaires).

- ▶ Pour préciser le système (binaire ou décimal) dans lequel est écrit un nombre, on utilise un indice (2 ou 10).
- ▶ Exemples :
 - 1011_2 est un nombre binaire
 - 1011_{10} ou 1011 est un nombre décimal.

► Dans une séquence de bits $b_{n-1}b_{n-2}...b_0$, on appelle :

- Bit de **poids faible**, le bit le plus à droite (b_0)
- Bit de **poids fort**, le bit le plus à gauche (b_{n-1})
- Bit de **poids p**, le bit b_p

► Par exemple, dans la séquence **101110010₂** :

- Le bit de poids faible est 0
- Le bit de poids fort est 1
- Le bit de poids 3 (ou de rang 3) est 0.

Multiples d'octets tel que définis par IEC 60027-2

Unités d'octets v · d · m						
Ordre de grandeur	Système international (SI)			Préfixes binaires		
	Unité	Notation	Valeur	Unité	Notation	Valeur
1	octet	o	1 octet	octet	o	1 octet
10^3	kiloctet	ko	10^3 octets	kibioctet	Kio	2^{10} octets
10^6	mégaoctet	Mo	10^6 octets	mébioctet	Mio	2^{20} octets
10^9	gigaoctet	Go	10^9 octets	gibioctet	Gio	2^{30} octets
10^{12}	téraoctet	To	10^{12} octets	tébioctet	Tio	2^{40} octets
10^{15}	pétaoctet	Po	10^{15} octets	pébioctet	Pio	2^{50} octets
10^{18}	exaoctet	Eo	10^{18} octets	exbioctet	Eio	2^{60} octets
10^{21}	zettaoctet	Zo	10^{21} octets	zébioctet	Zio	2^{70} octets
10^{24}	yottaoctet	Yo	10^{24} octets	yobioctet	Yio	2^{80} octets

Quelques ordres de grandeurs :



Un CD
700 Mo = 0.7Go



Un DVD
4.7 Go



Un Blu-Ray
25 Go



Une musique
4 Mo



Une photo
6 Mo



Un document
50 Ko



Un film
700 Mo



Un ordinateur récent
de 500 Go à 4 To



Une clé USB / carte mémoire
de 8 Go à 200 Go



Une disquette
1.4 Mo

- ▶ Combien de séquences binaires différentes peut-on écrire :
 - ▶ Avec 1 bit ? 2 séquences : 0 et 1
 - ▶ Avec 2 bits ? 4 séquences : 00, 01, 10 et 11
 - ▶ Avec 3 bits ? 8 séquences : 000, 001, 010, 011, 100, 101, 110, 111
 - ...
 - ▶ Avec n bits ? 2^N séquences
- ▶ L'utilisation d'un bit supplémentaire permet de doubler la capacité de codage.
- ▶ Chaque séquence binaire permet ensuite de coder une valeur parmi celles que l'on souhaite représenter (caractères, nombres, couleurs, ...).

- ▶ Inversement, pour trouver le nombre de bits nécessaires au codage de v valeurs, il faut trouver la plus petite puissance de 2 supérieure à v .
 - ▶ On obtient cette puissance avec l'opération $\log_2(v)$

Exemple du codage de l'alphabet latin :

- ▶ Pour représenter les 26 caractères de l'alphabet latin, il nous faut 5 bits.
 - ▶ Car $2^4 = 16 < 26$ et $2^5 = 32 \geq 26$
- ▶ On peut ensuite, par exemple, utiliser la représentation suivante :
 - ▶ 00000 → a, 00001 → b, 00010 → c ... 11001 → z

- Pour compter en binaire, c'est le même principe qu'en décimal :

0, 1, 2, 3, 4, 5, 6, 7, 8, 9 soit

0, 1, 10, 11, 100, 101, 110, 111, 1000, 1001

- On a donc :

- $1_2 = 1 = 2^0$

- $10_2 = 2 = 2^1$

- $100_2 = 4 = 2^2$

- $1000_2 = 8 = 2^3$

- Plus généralement, la valeur décimale associée au bit de poids n est 2^n .

- On en déduit donc la valeur décimale d'une suite binaire

$b_{n-1}b_{n-2}...b_0$.

$$b_{n-1} \times 2^{n-1} + b_{n-2} \times 2^{n-2} + \dots + b_0 \times 2^0 = \sum_{i=0}^{n-1} b_i \times 2^i$$

- Exemple :

$$1001101_2 = 1 \times 2^6 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^0 = 64 + 8 + 4 + 1 = 77$$

- ▶ On connaît maintenant le système binaire basé sur les bits et le système décimal basé sur les chiffres arabes.
- ▶ On appelle base B un système dans lequel on dispose d'un alphabet de B chiffres (ou symboles) pour écrire des nombres.
 - ▶ Système binaire $\rightarrow$ base 2
 - ▶ Système décimal $\rightarrow$ base 10
- ▶ Ce que nous avons vu pour le système binaire peut être généralisé :
 - ▶ $X_{n-1}X_{n-2} \dots X_0$ dans une base B vaut $X_{n-1}*B^{n-1} + X_{n-2}*B^{n-2} + \dots + X_0$ en décimal.
 - ▶ Dans une base B , n chiffres permettent de coder B^n valeurs différentes.
- ▶ Exemples :
 - ▶ $5672_8 = 5*8^3 + 6*8^2 + 7*8^1 + 2*8^0 = 3002$
 - ▶ En base 7, on peut coder, avec 3 chiffres, 7^3 soit 343 valeurs différentes.

B	Nom	Symboles	Exemple écriture de 93
2	binaire	0,1	1011101_2
8	octale	0,1,2,3,4,5,6,7	135_8
10	décimale	0,1,2,3,4,5,6,7,8,9	93_{10}
16	hexadécimale	0,1,2,3,4,5,6,7,8,9, A,B,C,D,E,F	$5D_{16}$

► Exemples d'utilisations :

- Binaire : ordinateur, langage machine
- Octale : droits sur un fichier linux (lecture, écriture, exécution)
- Décimal : adresses IPv4
- Hexadécimale : adresses mac, ipv6, ASCII ...

- ▶ Pour passer d'un nombre entier de **base 10 en base 2**, il suffit de faire des divisions successives par 2 jusqu'à obtenir un reste égal à 1 ou 0 donc inférieur à 2.
 - ▶ La suite des restes obtenus correspond au chiffre dans la base visée : $a_n a_{n-1} \dots a_1 a_0$
 - ▶ **Exemple** : $44_{10} \Leftrightarrow 101100_2$
- ▶ Une autre méthode (plus simple) consiste à soustraire au nombre à convertir la puissance binaire (1, 2, 4, 8, 16, 32, ...) immédiatement inférieure à ce nombre. Puis on procède de manière identique avec le reste jusqu'à obtenir 0.
 - ▶ **Exemple** : $75_{10} \Leftrightarrow 1001011_2$
- ▶ Pour le passage **base 2 vers base 10**, il suffit de décomposer le nombre en fonction des puissances binaires et d'additionner.
soit : $a_n * 2^n + a_{n-1} * 2^{n-1} + \dots + a_1 * 2^1 + a_0 * 2^0$
Exemple : $1111\ 1111_2 = 128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 255_{10}$

- ▶ Avec 4 bits, on peut coder 2^4 soit 16 valeurs (1 chiffre hexadécimal)
- ▶ De **base 2 vers base 16**, méthode:
 - ▶ On groupe les bits par 4 en commençant par les bits de poids faible.
 - ▶ On ajoute si nécessaire des bits de poids fort à 0 pour obtenir un nombre de bits multiple de 4.
 - ▶ On convertit chaque suite de 4 bits en hexadécimal comme suit:

Base 2	Base 16	Base 2	Base 16
0000	0	1000	8
0001	1	1001	9
0010	2	1010	A
0011	3	1011	B
0100	4	1100	C
0101	5	1101	D
0110	6	1110	E
0111	7	1111	F

► **Exemple :**

► 11100110001101101_2

► $1\ 1100\ 1100\ 0110\ 1101_2$

► $0001\ 1100\ 1100\ 0110\ 1101_2$

► **$1CC6D_{16}$**

- De **base 16 vers base 2** : On convertit chaque chiffre hexadécimal en son équivalent binaire (sur 4 bits). Une fois le nombre binaire trouvé, on ôte les zéros de poids fort.

► **Exemple :**

► $1B34D_{16} \Leftrightarrow 0001\ 1011\ 0011\ 0100\ 1101\ 0001_2$

► $1B34D_{16} \Leftrightarrow 110110011010011010001_2$

- ▶ Pour passer d'un nombre entier de **base 10 en base 16**:
 - ▶ il suffit de faire des divisions successives par 16 jusqu'à obtenir un reste inférieur à 15.
 - ▶ La suite des restes obtenus, convertit en base 16, correspond au chiffres dans la base visée : $a_n a_{n-1} \dots a_1 a_0$
 - ▶ **Exemple** : $5835(10) \Leftrightarrow 16CB(16)$
 - ▶ On peut aussi appliquer la méthode de la soustraction vue précédemment.
- ▶ Pour le passage **base 16 vers base 10**, il suffit de décomposer le nombre en fonction des puissances de la base 16 et d'additionner.
 - ▶ soit : $a_n * 16^n + a_{n-1} * 16^{n-1} + a_1 * 16^1 + a_0 * 16^0$
 - ▶ **Exemple** : $2F35(16) = 2 * 16^3 + 15 * 16^2 + 3 * 16^1 + 5 * 16^0$
$$= 2 * 4096 + 15 * 256 + 3 * 16 + 5$$
$$= 8192 + 3840 + 18 + 5$$
$$= 12085_{10}$$

- ▶ En plus des opérateurs arithmétique (+,-,x,/) les programmes ont parfois besoin d'effectuer des opérations logiques.
- ▶ On ne voit plus alors la séquence binaire comme un nombre mais comme une suite de valeurs **vrai** (1) ou **faux** (0).
- ▶ On utilise une table de vérité qui donne, en fonction des bits des opérandes, le bit résultant de l'opération.
- ▶ Nous allons voir 4 opérateurs logique: ET, OU, OU exclusif et NON.
- ▶ Exemples d'utilisation de ces opérateurs:
 - ▶ ET, OU et NON utilisés pour certains calculs sur les adresses Ipv4 (voir cours et TD)
 - ▶ OU exclusif utilisé par des algorithmes de cryptage.

- Le résultat donne VRAI si les variables d'entrée sont à VRAI.
- Table de vérité du ET (à 2 variables):

a	b	a ET b
0	0	0
0	1	0
1	0	0
1	1	1

- Exemple:

ET	1	1	0	1	1	0	0	1
	0	1	1	1	0	0	1	1
	0	1	0	1	0	0	0	1

- ▶ Au moins une des variable d'entrée doit être VRAI.
- ▶ Table de vérité du OU (à 2 variables):

a	b	a OU b
0	0	0
0	1	1
1	0	1
1	1	1

- ▶ Exemple:

$$\begin{array}{rcccccccc}
 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 1 \\
 \text{OU} & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 1 \\
 \hline
 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 1
 \end{array}$$

- Un nombre impair de variables d'entrée doivent être VRAI.
- Table de vérité du XOR (à 2 variables):

a	b	a XOR b
0	0	0
0	1	1
1	0	1
1	1	0

- Exemple:

$$\begin{array}{r}
 \text{XOR} \quad \begin{array}{cccccccc} 1 & 1 & 0 & 1 & 1 & 0 & 0 & 1 \end{array} \\
 \begin{array}{cccccccc} 0 & 1 & 1 & 1 & 0 & 0 & 1 & 1 \end{array} \\
 \hline
 \begin{array}{cccccccc} 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \end{array}
 \end{array}$$

► Inversion du bit de l'opérande.

► Table de vérité du NON:

a	NON a
0	1
1	0

► Exemple:

NON	0	1	1	1	0	0	1	1
	1	0	0	0	1	1	0	0

- ▶ Une **table de codage** associe une séquence binaire à chaque caractère de l'alphabet que l'on veut coder.
- ▶ L'ordinateur traduit ensuite chaque caractère du texte en une séquence de bits en utilisant cette table.
- ▶ Il y a différentes tables de codage.
- ▶ Elles se différencient par:
 - ▶ L'ensemble des caractères qu'elles permettent de coder
 - ▶ Et le nombre de bits qu'elles utilisent pour coder les caractères.
- ▶ Nous allons en voir 2: l' ASCII et l'UTF-8
- ▶ Pour connaître le codage utilisé pour un fichier:
file <nom-du-fichier>
- ▶ Exemple:

```
$ file message.txt  
message.txt: UTF-8 Unicode text
```


- ▶ ASCII = American Standard Code for Information Interchange
- ▶ Les caractères ASCII sont codés sur 7 bits
- ▶ Permet de coder : *les lettres de l'alphabet latin (minuscules et majuscules), les chiffres arabes, les caracteres de ponctuation et d'autres caracteres speciaux (ex : +, -, ESC (la touche Echap du clavier)).*
- ▶ La table des caractères ASCII:

Base 16		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
	Base 2	0000	001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
000	0000	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
001	0001	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
002	0010	SP	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
003	0011	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
004	0100	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
005	0101	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	}	^	_
006	0110	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
007	0111	p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL

- ▶ En ligne, les 3 bits de poids fort (0 à 7), en colonne, les 4 bits de poids faible (0 à F).
- ▶ Exemple: le caractère A est codé 100 0001 soit 41_{16} ou 65_{10}

- ▶ UTF-8 = Universal character set Transformation Format - 8 bits
- ▶ codage le plus utilisé dans les systemes Linux
- ▶ caracteres UTF-8 codés sur 1, 2, 3 ou 4 octets
- ▶ permet de coder des textes d'à peu près n'importe quel alphabet (arabe, chinois, grec, indien, latin, . . .)
- ▶ compatible avec l'ASCII : Un texte codé en ASCII est codé de la même maniere en UTF-8.
 - ▶ Les caracteres ASCII etant codés sur 7 bits on rajoute un bit de poids fort à 0 pour obtenir un caractere UTF-8 sur 1 octet.)
- ▶ Exemples de codage :

E -> 0100 0101 soit U+0045

É -> 1100 1001 soit U+00C9

π -> 0011 1100 0000 soit U+3C0

🕒 -> 1111 0000 1000 1011 soit U+F08B